

“Collaborative Story/Screenplay Writing Platform with Version History”

CO2 ASSESSMENT TOOL 1

Submitted in the partial fulfilment for the Course of

CSA1016 - Software Engineering

to the award of the degree of

BACHELOR OF

ENGINEERING IN

Artificial Intelligence & Data Science

Submitted by

192424169- LITHISH KUMAR. R

Under the

Supervision of Dr. K

Anita Davamani



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**

August 2025



SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**



DECLARATION

I, **Lithish Kumar R 192424179**, of the **Department of Artificial Intelligence and Data Science**, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled **“Collaborative Story/Screenplay Writing Platform with Version History”** is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place:

Chenna

i Date:

Signature of the Students

TABLE OF CONTENTS

Sl. No.	Section	Page No.
1	Introduction	3
2	Problem Description	3
3	Scope of the System	3
4	Stakeholders, Business Domain and System Objectives	4
5	Functional Requirements	4
6	Non-Functional Requirements	5
7	Actors and Use Cases	6
	7.1 Primary and Secondary Actors	6
	7.2 Use Case Diagram	6
	7.3 Use Case Descriptions	7
8	Data Requirements	7
9	External Interface Requirements	8
10	System Features	8
11	System Architecture (Supporting Diagram)	9
12	Assumptions	9
13	Constraints	9
14	Requirement Validation — Completeness and Consistency	10
15	Conclusion	10

1. Introduction

The Collaborative Story/Screenplay Writing Platform with Version History is a cloud-based, web-accessible software system designed to enable writers, co-writers, and editors to jointly author stories, novels, or screenplays in real time. The system tracks every change made to a document through an integrated version-control mechanism, allowing users to view historical revisions, compare differences between versions, and restore earlier drafts whenever required.

This Software Requirement Specification (SRS) document describes the functional and non-functional requirements, actors, use cases, data requirements, external interfaces, system features, assumptions, and constraints of the proposed platform, following a scenario-based requirement analysis approach.

2. Problem Description

Writers working on stories or screenplays frequently collaborate with co-authors, editors, and reviewers. Traditional file-sharing methods (email attachments, shared folders, generic word processors) create serious challenges:

- Multiple conflicting copies of the same script circulate among collaborators, causing confusion over which version is final.
- There is no reliable way to track who changed what, when, and why.
- Reverting to an earlier draft after an unwanted edit is difficult or impossible.
- Real-time simultaneous editing is not supported, forcing writers to work sequentially and wait for turns.
- Feedback and review comments get lost across email threads instead of being attached to the specific scene or line they refer to.
- Exporting a finished script into industry-standard formats (e.g., PDF, Fountain) is a manual, error-prone process.

The proposed system addresses these issues by providing a single collaborative workspace with built-in, granular version history, real-time co-editing, inline commenting, and controlled access for every stakeholder in the writing process.

3. Scope of the System

The system scope covers the complete authoring lifecycle of a story or screenplay: project creation, real-time collaborative writing, scene and character management, automatic and manual version snapshots, difference (diff) comparison between versions, restoration of previous versions, inline review comments, role-based access control, notifications, and export to standard formats.

The system is a web-based (and optionally mobile-accessible) application. It does not cover offline desktop word-processing features unrelated to collaborative writing, nor does it include film production, budgeting, or scheduling functionality, which lie outside the boundary of this platform.

4. Stakeholders, Business Domain and System Objectives

4.1 Business Domain

The system operates in the creative writing and media pre-production domain, supporting individual writers, writing teams, small production houses, publishing houses, and educational creative-writing programs.

4.2 Stakeholders

- Writers/Authors — create and edit original content.
- Co-Writers/Collaborators — jointly edit shared projects.
- Editors/Reviewers — review, comment on, and approve changes.
- System Administrators — manage accounts, permissions, and system health.
- Guest Readers — view read-only shared drafts (e.g., producers, clients).
- Product Owner/Organization — owns the platform and its business goals.

4.3 System Objectives

- Provide a single source of truth for collaborative scripts with complete version traceability.
- Enable real-time, conflict-free simultaneous editing by multiple users.
- Allow any prior version to be viewed, compared, and restored without data loss.
- Support structured review workflows through inline comments and change approval.
- Ensure secure, role-based access to sensitive creative content.
- Provide reliable export to industry-standard screenplay and document formats.

5. Functional Requirements

ID	Functional Requirement	Description
FR-01	User Registration & Login	The system shall allow users to register, log in, and manage their profile using secure authentication (including optional multi-factor authentication).
FR-02	Project/Script Creation	The system shall allow a Writer to create a new story/screenplay project and define its title, genre, and format (novel, screenplay, stage play).
FR-03	Real-Time Collaborative Editing	The system shall allow multiple authorized users to edit the same document simultaneously with live cursor presence and automatic conflict resolution.
FR-04	Scene & Character Management	The system shall allow users to create, organize, and reorder scenes, and maintain a character list linked to the script.
FR-05	Automatic Version Snapshots	The system shall automatically save a new version snapshot at defined intervals or after significant edits.
FR-06	Manual Version Save	The system shall allow a user to manually save and label a named version/checkpoint (e.g., 'Draft 3 — Act 2 revised').
FR-07	Version History Viewer	The system shall display a chronological list of all saved versions with author, timestamp, and change summary.
FR-08	Version Comparison (Diff)	The system shall allow users to select any two versions and view a line-by-line difference (added/removed/modified text).
FR-09	Version Restore/Rollback	The system shall allow an authorized user to restore the document to any previous version without permanently deleting later versions.
FR-10	Inline Commenting	The system shall allow reviewers to attach comments and suggestions to specific lines, scenes, or characters.

ID	Functional Requirement	Description
FR-11	Change Approval Workflow	The system shall allow designated Editors to approve or reject proposed changes before they are merged into the main draft.
FR-12	Role-Based Access Control	The system shall support roles (Owner, Co-Writer, Editor, Guest) with distinct permissions for editing, commenting, and viewing.
FR-13	Notifications	The system shall notify users of comments, mentions, approvals, and merge events via in-app and email notification.
FR-14	Export	The system shall allow export of the script to PDF, DOCX, and Fountain/screenplay-standard formats.
FR-15	Search	The system shall allow full-text search across scenes, characters, comments, and version history.
FR-16	Sharing & Invitations	The system shall allow project owners to invite collaborators via email/link with a defined role.
FR-17	Administration Console	The system shall allow Administrators to manage user accounts, monitor usage, and configure system-wide settings.

6. Non-Functional Requirements

Category	Requirement
Performance	Real-time edits by any collaborator shall be reflected on all connected clients within 500 milliseconds under normal network conditions.
Scalability	The system shall support at least 10,000 concurrent active projects and horizontal scaling of application servers during peak load.
Security	All data in transit shall be encrypted using TLS 1.2+ and data at rest shall be encrypted using AES-256; access shall be governed by role-based permissions.
Reliability	The system shall guarantee 99.9% uptime and shall not lose committed version data even in the event of a server failure (durable storage/backups).
Usability	The editing interface shall follow a familiar word-processor/screenplay-editor layout, usable by non-technical writers with minimal training.
Maintainability	The system shall follow a modular, service-oriented architecture to allow independent updates to the editor, version-control engine, and notification service.
Availability	Core editing and version-history features shall remain available 24/7, with scheduled maintenance windows communicated to users in advance.
Portability	The web client shall function consistently across major modern browsers (Chrome, Firefox, Edge, Safari) and responsive mobile layouts.
Auditability	Every version change and access-control modification shall be logged with user ID, timestamp, and action performed.

7. Actors and Use Cases

7.1 Primary and Secondary Actors

Actor	Type	Description
Writer/Author	Primary	Creates and edits scripts; owns projects; manages sharing and versions.
Co-Writer/Collaborator	Primary	Jointly edits shared scripts in real time; views and compares versions.
Editor/Reviewer	Primary	Reviews drafts, adds comments/suggestions, approves or rejects merges.
System Administrator	Primary	Manages user accounts, access permissions, and monitors system health.
Guest Reader	Secondary	Views shared scripts in read-only mode; cannot edit.
Notification Service	Secondary (System Actor)	Automated component that dispatches in-app and email notifications.

7.2 Use Case Diagram

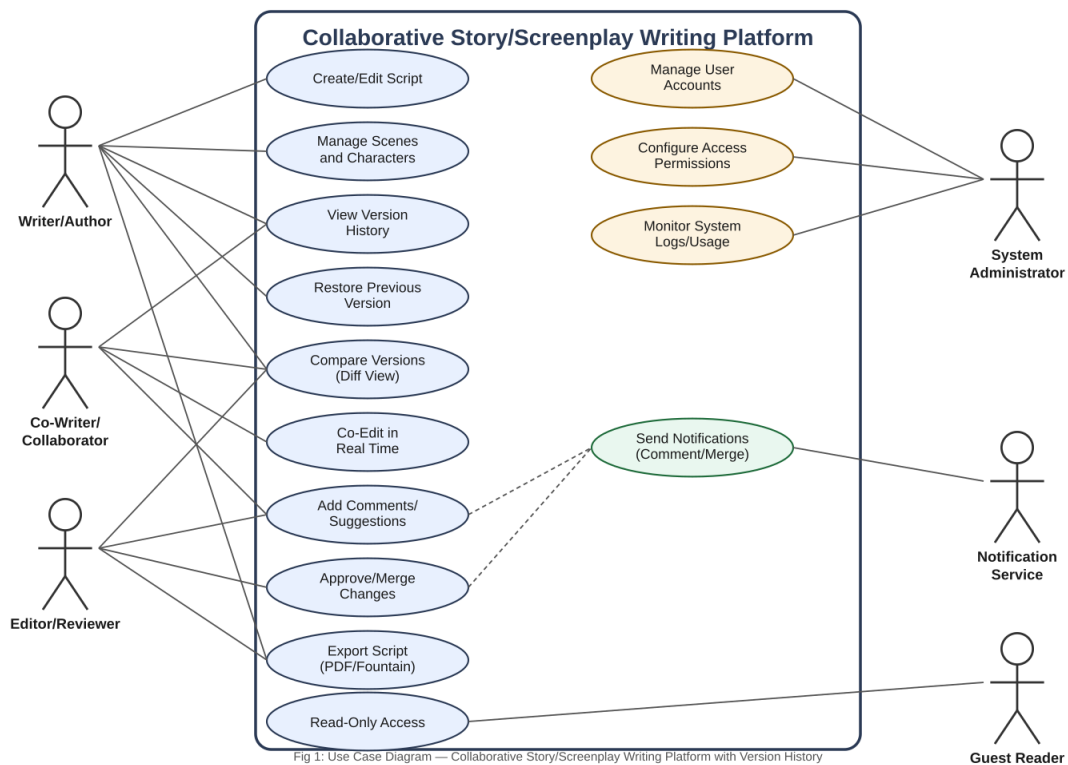


Fig. 1 — Use Case Diagram: Collaborative Story/Screenplay Writing Platform with Version History

7.3 Use Case Descriptions

Use Case	Primary Actor	Description
Create/Edit Script	Writer/Co-Writer	Actor creates a new script or edits content within an existing project.
Manage Scenes and Characters	Writer/Co-Writer	Actor adds, reorders, or removes scenes and maintains the character list.

Use Case	Primary Actor	Description
View Version History	Writer/Co-Writer/ Editor	Actor views a chronological list of saved versions with metadata.
Compare Versions (Diff)	Writer/Co-Writer/ Editor	Actor selects two versions and views the highlighted differences.
Restore Previous Version	Writer/Co-Writer	Actor rolls the current draft back to a previously saved version.
Co-Edit in Real Time	Co-Writer	Actor edits the same document concurrently with other collaborators.
Add Comments/Suggestions	Editor/Reviewer	Actor attaches review feedback to specific lines or scenes.
Approve/Merge Changes	Editor/Reviewer	Actor reviews proposed edits and approves or rejects the merge.
Export Script	Writer/Editor	Actor exports the finished script to PDF, DOCX, or Fountain format.
Read-Only Access	Guest Reader	Actor views a shared script without edit permissions.
Manage User Accounts / Access	Administrator	Actor creates, disables, or configures roles for user accounts.
Monitor System Logs/Usage	Administrator	Actor reviews audit logs and system usage/performance metrics.

8. Data Requirements

- User Data: user ID, name, email, hashed password/auth token, role, profile details.
- Project Data: project ID, title, genre, format, owner ID, creation date, collaborator list.
- Document Content: current script text/content tree (scenes, characters, dialogue blocks).
- Version Records: version ID, parent version ID, author ID, timestamp, change summary, full snapshot or diff patch.
- Comment Data: comment ID, author, target line/scene reference, text, status (open/resolved), timestamps.
- Access Control Data: role assignments, permission scopes, invitation status per project.
- Notification Data: event type, recipient, delivery channel, read/unread status.
- Audit Log Data: action type, actor, target object, timestamp, IP/session metadata.

9. External Interface Requirements

9.1 User Interfaces

- Responsive web application with a script/story editor, version-history sidebar, comment panel, and diff viewer.

9.2 Hardware Interfaces

- Standard client devices (desktop, laptop, tablet, smartphone) with a modern web browser and internet connectivity.

9.3 Software Interfaces

- Authentication provider (OAuth 2.0 / SSO integration for institutional or organizational login).
- Email/SMS gateway for notifications.

- Cloud object storage service for exported files and attachments.
- Third-party export libraries/services for Fountain and PDF generation.

9.4 Communication Interfaces

- HTTPS/REST APIs for standard client-server operations.
- WebSocket-based protocol for real-time collaborative editing and presence updates.

10. System Features

Feature	Priority	Summary
Real-Time Co-Editing	High	Multiple users edit the same script concurrently with live presence indicators.
Version History & Rollback	High	Automatic and manual snapshots with the ability to view, compare, and restore any version.
Inline Review & Approval	High	Comment threads and a merge-approval workflow for controlled collaboration.
Role-Based Access Control	High	Owner, Co-Writer, Editor, and Guest roles with distinct permissions.
Export Engine	Medium	One-click export to PDF, DOCX, and Fountain screenplay format.
Notifications	Medium	In-app and email alerts for comments, mentions, approvals, and merges.
Full-Text Search	Medium	Search across script content, comments, and version metadata.
Admin Console	Low	Account management, usage monitoring, and system configuration for administrators.

11. System Architecture (Supporting Diagram)

The following layered architecture diagram illustrates how the client, application, data, and infrastructure layers interact to deliver real-time collaborative editing with version control.

System Architecture Overview

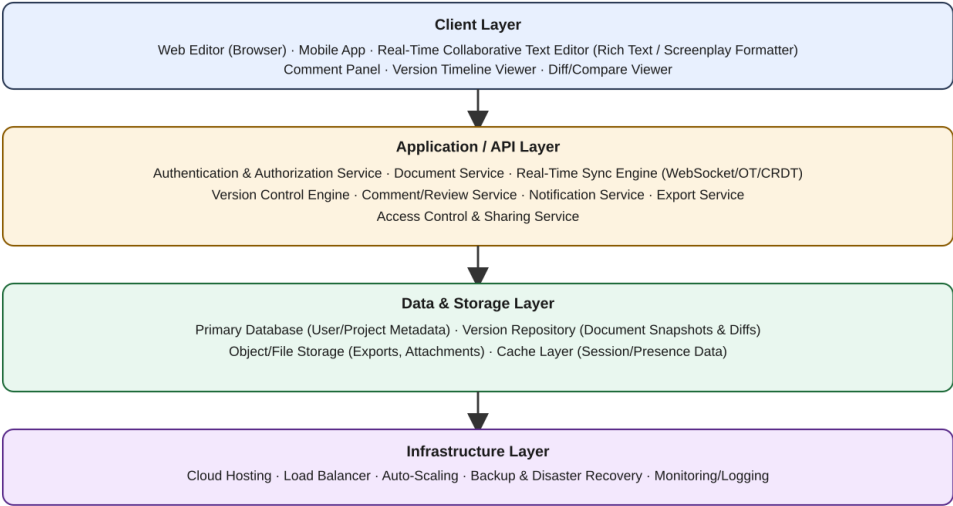


Fig. 2 — High-Level Layered System Architecture

12. Assumptions

- Users have consistent access to a stable internet connection while using real-time collaborative features.
- Each writing project has at least one designated Owner responsible for managing collaborators.
- Browser clients support modern JavaScript/WebSocket capabilities required for live editing.
- Users are responsible for the accuracy of the creative content they author; the system only manages structure and versioning.
- Third-party authentication and storage providers maintain their own agreed service-level agreements (SLAs).

13. Constraints

Type	Constraint
Technical	Real-time synchronization requires a conflict-resolution algorithm (OT/CRDT) that must handle concurrent edits without data loss.
Operational	The system must remain available during peak collaborative writing sessions across multiple time zones.
Economic	Cloud storage and compute costs scale with the number of stored versions, requiring a version-retention/archival policy.
Legal	The system must comply with data-protection regulations (e.g., GDPR) for user data and intellectual-property protection for creative content.
Time-Related	The initial release must prioritize core editing and version-control features within the project timeline before advanced analytics or AI features.

14. Requirement Validation — Completeness and Consistency

14.1 Stakeholder Coverage Check

Each identified stakeholder (Writer, Co-Writer, Editor, Administrator, Guest Reader) is mapped to at least one functional requirement and one use case, confirming that no stakeholder's core need has been left unaddressed.

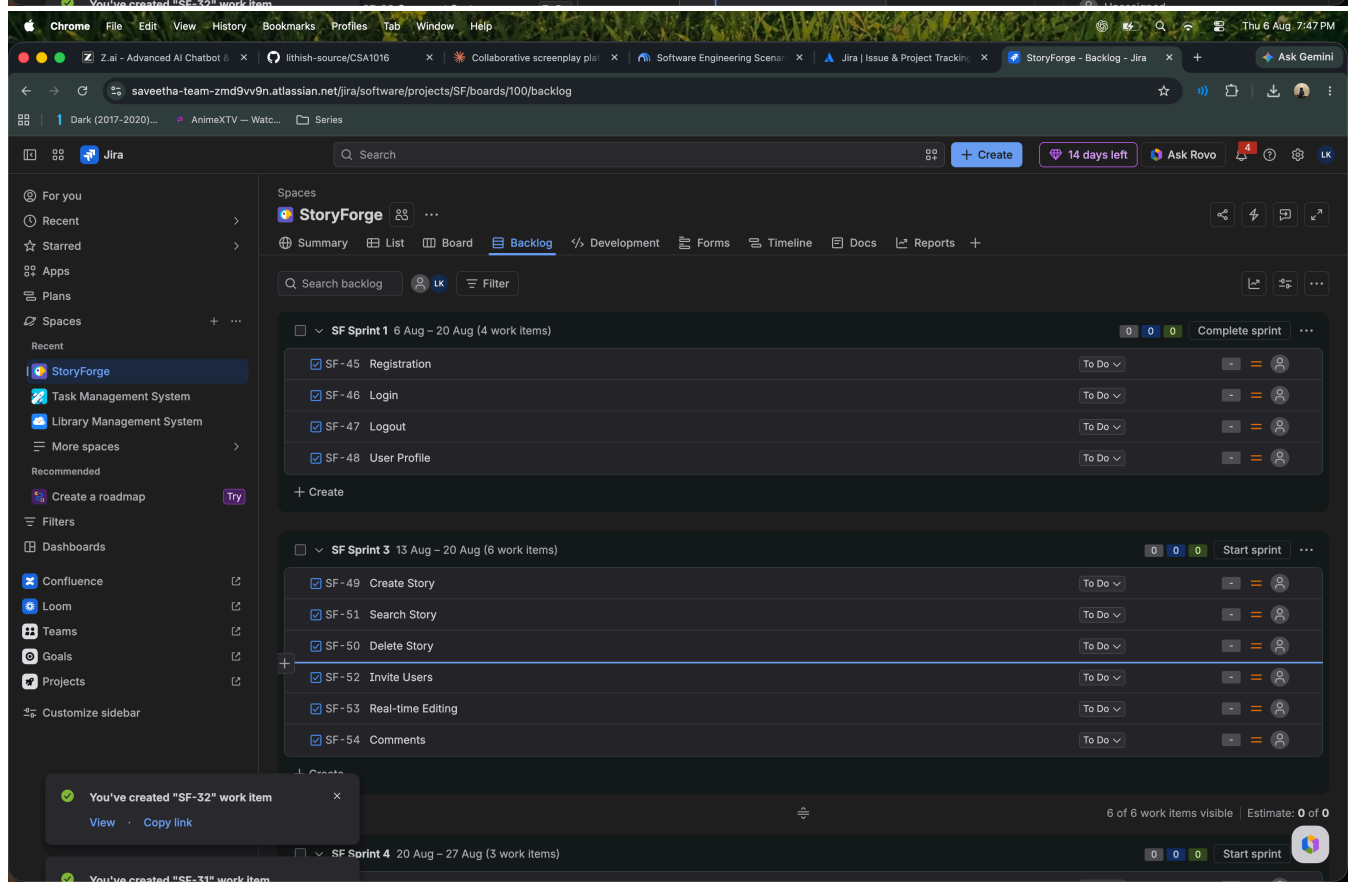
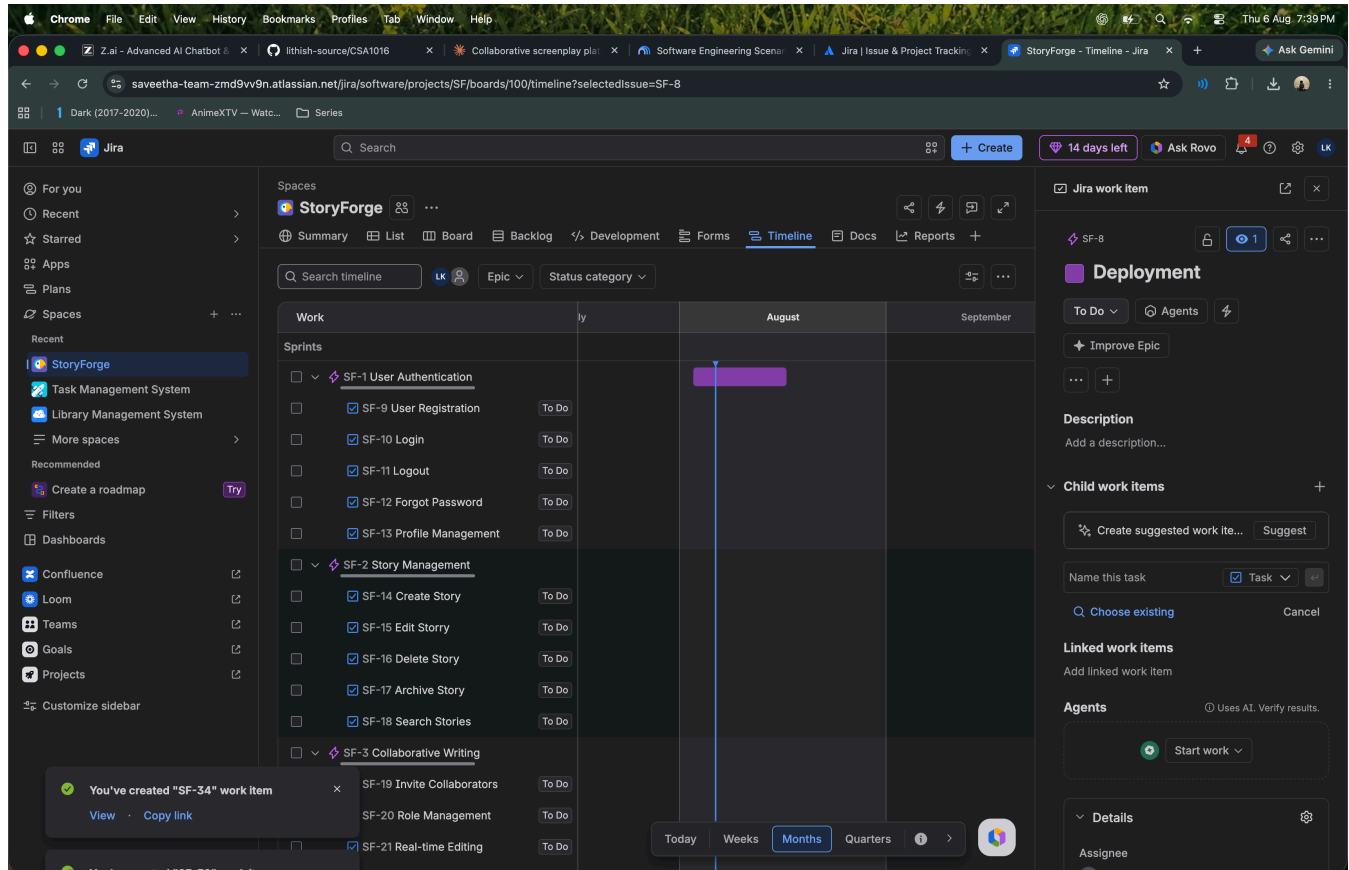
14.2 Ambiguity, Redundancy and Inconsistency Review

- Ambiguity: Terms such as 'significant edits' (FR-05) and 'peak load' (Scalability NFR) are flagged as needing quantified thresholds in the next design iteration (e.g., defining 'significant edit' as 50+ changed characters).
- Redundancy: FR-07 (Version History Viewer) and FR-08 (Compare Versions) were reviewed to confirm they serve distinct purposes (listing vs. differencing) rather than duplicating functionality.
- Inconsistency: Access-control terminology was standardized across FR-12, the Actors table, and the Non-Functional Security requirement to consistently use the roles Owner, Co-Writer, Editor, and Guest.
- Missing Requirements Identified During Review: an explicit conflict-resolution requirement for simultaneous edits (added under FR-03) and an audit-logging requirement (added under Data Requirements and Auditability NFR) were included after the initial pass.

14.3 Suggested Improvements

- Quantify vague terms (e.g., define exact auto-save intervals and concurrency limits) in the detailed design phase.
- Add explicit acceptance criteria for each functional requirement to support test-case generation.
- Introduce a formal requirement traceability matrix linking each FR/NFR to its corresponding use case and test case.
- Conduct a stakeholder walkthrough/review session before moving to the design phase to validate assumptions.

16. Jira Screenshots:



The screenshot shows the Jira StoryForge interface for the 'SF-46 Login' issue. The main view is the 'Backlog' tab, displaying a list of work items for 'SF Sprint 1' (6 Aug - 20 Aug) and 'SF Sprint 3' (13 Aug - 20 Aug). The 'SF-46 Login' issue is highlighted in the backlog. The right sidebar shows the 'Jira work item' details for 'SF-46 Login', including a 'Login' title, a 'To Do' status, and a 'Subtasks' section. The 'Subtasks' section lists several tasks: 'Design Login UI', 'Implement Login UI', 'Validate Create...', 'Test Login', and 'Test Login'. The 'Status' column for all subtasks is 'To Do'. A notification at the bottom left states 'You've created "SF-64" work item'.

Spaces

StoryForge

Summary List Board Backlog Development Forms Timeline Docs Reports

Search backlog Filter

SF Sprint 1 6 Aug - 20 Aug (4 work items) Complete sprint

- SF-45 Registration To Do
- SF-46 Login To Do
- SF-47 Logout To Do
- SF-48 User Profile To Do

+ Create

SF Sprint 3 13 Aug - 20 Aug (6 work items) Start sprint

- SF-49 Create Story To Do
- SF-51 Search Story To Do
- SF-50 Delete Story To Do
- SF-52 Invite Users To Do
- SF-53 Real-time Editing To Do
- SF-54 Comments To Do

+ Create

SF Sprint 4 20 Aug - 27 Aug (3 work items) Start sprint

6 of 6 work items visible | Estimate: 0 of 0

You've created "SF-64" work item
View Copy link

You've created "SF-63" work item

You've created "SF-61" work item

Jira work item

Add epic / SF-46

Login

To Do Agents

Improve Task

Description

Add a description...

Subtasks

Create suggested work item... Suggest

0% Done

Work	P...	A...	Status
SF-61 Design Login UI	=		To Do
SF-62 Implement Login UI	=		To Do
SF-63 Validate Create...	=		To Do
SF-64 Test Login	=		To Do

Name this subtask Subtask

Choose existing Cancel

Linked work items

This screenshot shows the same Jira StoryForge interface, but with additional subtasks added to the 'SF-46 Login' issue. The 'Subtasks' section now includes 'Design Login UI', 'Implement Login UI', 'Validate Create...', 'Test Login', and 'Test Login'. The 'Status' column for all subtasks is 'To Do'. A notification at the bottom left states 'You've created "SF-64" work item'.

Spaces

StoryForge

Summary List Board Backlog Development Forms Timeline Docs Reports

Search backlog Filter

SF Sprint 1 6 Aug - 20 Aug (4 work items) Complete sprint

- SF-45 Registration To Do
- SF-46 Login To Do
- SF-47 Logout To Do
- SF-48 User Profile To Do

+ Create

SF Sprint 3 13 Aug - 20 Aug (6 work items) Start sprint

- SF-49 Create Story To Do
- SF-51 Search Story To Do
- SF-50 Delete Story To Do
- SF-52 Invite Users To Do
- SF-53 Real-time Editing To Do
- SF-54 Comments To Do

+ Create

SF Sprint 4 20 Aug - 27 Aug (3 work items) Start sprint

6 of 6 work items visible | Estimate: 0 of 0

You've created "SF-64" work item
View Copy link

You've created "SF-63" work item

You've created "SF-61" work item

Jira work item

Add epic / SF-46

Login

To Do Agents

Improve Task

Description

Add a description...

Subtasks

Create suggested work item... Suggest

0% Done

Work	P...	A...	Status
SF-61 Design Login UI	=		To Do
SF-62 Implement Login UI	=		To Do
SF-63 Validate Create...	=		To Do
SF-64 Test Login	=		To Do

Name this subtask Subtask

Choose existing Cancel

Linked work items

